Diseño y Desarrollo de Servicios Web: Registro e Inicio de Sesión

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

La evidencia pide un servicio web que reciba usuario y contraseña y devuelva «autenticación satisfactoria» o un error de autenticación. Dicho así parece una tarde de trabajo. Lo que convierte el ejercicio en algo serio es que **la lectura literal del enunciado produce un servicio inseguro**: si el error distingue «ese correo no existe» de «la contraseña no coincide», el endpoint deja de ser un inicio de sesión y pasa a ser un buscador de cuentas registradas.

Este documento explica el servicio construido —`autenticacion-pergamino`, un módulo Spring Boot aparte— y, sobre todo, las decisiones que no se ven en el código terminado: qué se cuenta, qué se calla, y por qué dos fallos reales no aparecieron hasta que el servicio se probó por HTTP de verdad.

2. Objetivo

Diseñar y codificar un servicio web de registro e inicio de sesión que autentique a los usuarios de MONK GRAIN, con el código comentado, los estándares de AA1-EV02 y control de versiones en Git.

3. Por qué un módulo aparte

La autenticación es lo único que todos los demás módulos necesitan. Mezclarla con el dominio del café obligaría a arrancar el catálogo entero para probar un inicio de sesión, y a desplegar el catálogo cada vez que cambie una regla de contraseñas.

Módulo	Evidencia	Puerto	Qué resuelve
`catalogo-pergamino`	AA2-EV01	—	Acceso a datos con JDBC
`catalogo-web`	AA2-EV02	8080	Pantallas con servlets y JSP
`negociacion-pergamino`	AA3-EV01	8081	Solicitudes de compra y posturas
`autenticacion-pergamino`	**AA5-EV01**	**8082**	**Registro e inicio de sesión**

Los cuatro usan la base `monkgrain_java` del mismo PostgreSQL del proyecto, **no** la de la aplicación: un módulo de aprendizaje que crea y borra cuentas no debe acercarse a los datos del producto.

4. Los cuatro endpoints

Método y ruta	Qué hace	Éxito
`POST /api/auth/register`	Crea la cuenta	202
`POST /api/auth/login`	Autentica y abre sesión	200
`GET /api/auth/me`	Devuelve la cuenta del token	200
`POST /api/auth/logout`	Cierra la sesión	200

Los dos últimos no los pide la evidencia. Están porque sin ellos el token sería un adorno: `/me` es lo que demuestra que la sesión quedó abierta de verdad, y no solo que el mensaje decía que sí.

5. Las decisiones de seguridad

5.1 La contraseña no se guarda

Lo que se guarda es su resumen BCrypt. **BCrypt y no SHA-256 ni MD5**, y el motivo es contraintuitivo: aquí la rapidez es un defecto. Una tarjeta gráfica calcula miles de millones de SHA-256 por segundo, así que quien robe la tabla prueba el diccionario entero en minutos. BCrypt está hecho para ser lento a propósito, con un factor de coste configurable.

```
$2a$10$7qRk0.X8vJ2mN4pQ...
 |   |  └── sal, 22 caracteres, distinta en cada cuenta
 |   └─────── factor de coste: 2^10 vueltas ≈ 100 ms
 └─────────── algoritmo
```

La sal va dentro del resultado, así que dos personas con la misma contraseña no comparten resumen. Sin ella, quien robara la tabla sabría de un vistazo qué cuentas comparten contraseña, y rompería todas a la vez.

> El producto usa **Argon2id**, que es la recomendación actual. Este módulo usa BCrypt porque es lo que trae `spring-security-crypto` sin añadir dependencias; Argon2 en Java exige BouncyCastle. La diferencia se declara en lugar de callarla: BCrypt con factor 10 es sólido y sigue siendo aceptable, pero no es lo que se elegiría hoy partiendo de cero.

5.2 El error no dice por qué

Esta es la decisión central, y es la que contradice la lectura literal del enunciado. Hay una sola excepción — `InvalidCredentialsException` — para los dos casos:

```
POST /api/auth/login   {"email":"ana@finca.co",  "password":"equivocada-123"}
→ 401  {"error":"authentication_failed","message":"Correo o contraseña incorrectos."}

POST /api/auth/login   {"email":"nadie@finca.co","password":"pergamino2026"}
→ 401  {"error":"authentication_failed","message":"Correo o contraseña incorrectos."}
```

Idénticos. Si se diferenciaran, probar un correo con cualquier contraseña diría si esa persona tiene cuenta en MONK GRAIN —un dato que en un mercado de café tiene valor comercial, porque revela quién está vendiendo—. Por lo mismo responde **401 y no 404**: un 404 diría «ese usuario no está aquí», que es exactamente lo que se calla.

5.3 El reloj también habla

Con los mensajes igualados queda una filtración menos obvia. Si el correo no existe, el servicio responde en un milisegundo; si existe, gasta los cien milisegundos de BCrypt comparando. Esa diferencia se mide desde fuera: basta cronometrar las respuestas para separar los correos registrados de los que no, aunque el mensaje sea idéntico.

```
if (found.isEmpty()) {
    // Se compara contra un resumen inventado para gastar el mismo
    // tiempo que si la cuenta existiera. El resultado se descarta.
    passwordEncoder.matches(password, dummyHash);
    throw new InvalidCredentialsException();
}
```

Medido sobre el servicio en marcha, doce peticiones de cada clase:

```
correo REGISTRADO, contraseña mala:   124,5 ms
correo INEXISTENTE:                   119,6 ms   ← 4 % de diferencia

campo faltante (no llega a BCrypt):     4,9 ms   ← esto es lo que se vería
                                                   sin el resumen inventado
```

La última línea es la que da la medida del problema: una respuesta que no pasa por BCrypt tarda **veinticinco veces menos**. Sin igualar el tiempo, separar los correos registrados de los que no sería tan fácil como cronometrar.

5.4 El registro tampoco delata

Un registro que respondiera «ese correo ya está en uso» reabriría por la puerta de atrás lo que se cerró en el ingreso. Así que responde lo mismo exista o no la cuenta:

```
POST /api/auth/register   correo nuevo
→ 202  {"message":"Si el correo no estaba registrado, tu cuenta quedó creada. Ya puedes iniciar sesión."}

POST /api/auth/register   el MISMO correo, con otro nombre y otra contraseña
→ 202  {"message":"Si el correo no estaba registrado, tu cuenta quedó creada. Ya puedes iniciar sesión."}
```

Y por eso responde **202 y no 201**: un 201 anuncia «se creó un recurso», y la gracia de esta respuesta es no decir si se creó. Con 201 para uno nuevo y otra cosa para uno repetido, el código de estado delataría lo que el mensaje calla. Una prueba compara los dos cuerpos byte por byte.

No es gratis. Un usuario despistado se queda sin saber por qué «no se creó» su cuenta. El precio se paga en el correo de confirmación, que es donde se le explica. Es la misma decisión que tomó el producto en `auth/router.py`.

5.5 El freno, y a quién se le explica

Cinco intentos fallidos seguidos frenan la cuenta quince minutos. El contador vive en la fila de la cuenta y no en memoria, para que el freno sobreviva a un reinicio del servicio: si no, bastaría esperar un despliegue para volver a empezar.

El punto fino es **el orden de las comprobaciones**. Primero se verifica la contraseña; después se mira si la cuenta está frenada:

Contraseña	Cuenta	Respuesta	Por qué
Equivocada	Libre	401 neutral	No se sabe si el correo existe
Equivocada	Frenada	**401 neutral**	Anunciar el freno confirmaría que el correo existe
Correcta	Frenada	**429 +** `Retry-After`	Quien acierta ya demostró ser el dueño
Correcta	Libre	200 + token	—

Al revés —mirando el freno antes de la contraseña— un atacante sabría que el correo existe con solo provocar el freno, sin acertar nunca la contraseña.

El código es **429 y no 403**: el 403 dice «no puedes, y no insistas», mientras que el 429 dice «ahora no, vuelve después», que es la verdad. La cabecera `Retry-After` va en segundos porque así la entienden los clientes que reintentan solos, sin leer el mensaje.

> **El freno tiene un costo que conviene nombrar.** Permite molestar a alguien concreto durante quince minutos, con solo fallar cinco veces contra su correo. Por eso insistir no alarga el freno —si cada intento lo extendiera, se podría dejar a un vendedor fuera indefinidamente— y por eso el producto no usa este mecanismo sino dos topes en Redis, uno por cliente y otro por cuenta, que retrasan sin cerrar. Aquí se implementa el freno porque es lo que se sostiene sin añadir Redis, y el compromiso queda escrito en el código en lugar de callarse.

5.6 Sesión revocable en vez de JWT

La elección más discutible del módulo, y la que más vale explicar. Un JWT no se puede revocar: lleva la firma dentro, así que cualquier servicio lo da por bueno hasta que caduque, aunque el usuario haya cerrado sesión o le hayan robado el teléfono. Para invalidarlo haría falta igualmente una lista de rechazados en la base —que es exactamente lo que el JWT prometía evitar—.

Con un identificador opaco contra una tabla, cerrar sesión es borrar una fila. El precio es una consulta por petición; a la escala de MONK GRAIN no se nota, y poder cerrar una sesión robada sí.

En la tabla se guarda el **resumen SHA-256 del token, no el token**: quien se lleve una copia de la base no debe quedarse con sesiones vivas en la mano. Aquí sí sirve SHA-256 y no hace falta BCrypt, porque el token lo genera el servidor con 256 bits de azar y no hay diccionario que probar contra él.

```
byte[] bytes = new byte[32];
random.nextBytes(bytes);   // SecureRandom, no Random:
                           // el segundo es predecible a partir de unas
                           // pocas salidas, así que quien viera un token
                           // podría calcular los siguientes.
return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
```

5.7 Lo que no sale en las respuestas

Las respuestas usan objetos propios y nunca la entidad. `AccountResponse` no lleva `passwordHash` —devolver la entidad publicaría el resumen en cada respuesta— ni `failedAttempts`, que le diría a un atacante cuántos intentos le quedan antes de que la cuenta se frene.

En la entrada, el mismo principio al revés: **lo que no está en el objeto de petición, no entra**. Una petición que pida ser administrador se ignora:

```
POST /api/auth/register
{"email":"ana@finca.co","password":"pergamino2026","fullName":"Ana Restrepo",
 "role":"ADMIN","id":1}

→ 202, y GET /api/auth/me devuelve  "role": "BUYER"
```

Es la lección de AA3-EV01, donde recibir la entidad como cuerpo permitía sobrescribir la postura ajena número 5 enviando `{"id": 5}`.

6. Los dos fallos que solo encontró el HTTP real

Esta sección es la más útil del documento. Las cuarenta pruebas estaban verdes y el servicio tenía dos defectos graves. Los dos se escondían por el mismo motivo: **`@Transactional` en la clase de prueba**.

6.1 `GET /me` devolvía 500

```
$ curl -s localhost:8082/api/auth/me -H "Authorization: Bearer $TOKEN"
{"error":"internal_error","message":"Ocurrió un error inesperado. Inténtalo de nuevo."}
HTTP 500

# en la bitácora del servidor:
org.hibernate.LazyInitializationException: could not initialize proxy
    [co.monkgrain.authentication.model.Account#1] - no Session
```

El servicio devolvía la cuenta como sustituto perezoso; su transacción terminaba ahí, y cuando el controlador leía el correo ya no había sesión de Hibernate abierta. En las prue-

bas, el `@Transactional` de la clase mantenía la sesión abierta durante todo el método y el sustituto se resolvía sin problema. Dieciséis pruebas verdes sobre un endpoint que no funcionaba.

Se arregló pidiendo la cuenta en la misma consulta:

```
@Query("select s from AuthSession s join fetch s.account where s.tokenHash = :tokenHash")
Optional<AuthSession> findByTokenHash(@Param("tokenHash") String tokenHash);
```

No con `FetchType.EAGER` en la entidad, que obligaría a traer la cuenta también al cerrar sesión, donde solo hace falta borrar la fila. Y **no** con `open-in-view=true`, el atajo habitual: mantener la sesión abierta durante toda la petición esconde este problema y, de paso, cualquier consulta accidental que haga una plantilla.

6.2 El freno no frenaba nada

Peor que el anterior, porque no se veía: el servicio respondía 401 correctamente, sumaba el intento… y lo perdía. Spring deshace la transacción cuando el método lanza una excepción no comprobada, y eso incluye el `InvalidCredentialsException` de un intento fallido. El contador subía en memoria y se borraba al salir, así que **la cuenta no se frenaba nunca** y toda la sección 5.5 era decorativa.

Las pruebas transaccionales tampoco lo veían: leían el contador desde la misma sesión de Hibernate donde seguía sumado.

```
@Transactional(noRollbackFor = InvalidCredentialsException.class)
public LoginResult login(String email, String password, Instant now) {
```

6.3 La prueba que sí los ve

Los dos fallos aparecieron con `curl`, no con `mvn test`. Para que no vuelva a pasar se añadió `AuthenticationHttpTest`, que reproduce las condiciones de producción: servidor real, puerto real y **sin transacción alrededor**. La limpieza se hace a mano antes de cada prueba, porque no hay nada que la deshaga.

Es la lección más transferible de esta evidencia: una prueba que pasa dentro de una transacción no demuestra que el endpoint funcione fuera de ella.

> Montar esa clase costó un tropiezo más. El cliente HTTP de serie de Java, `HttpURLConnection`, al recibir un 401 intenta por su cuenta repetir la petición con credenciales, y como el cuerpo ya salió por el socket falla con `cannot retry due to server authentication` en lugar de entregar la respuesta. Como en este servicio casi todo lo interesante responde 401, no se podía probar nada con él: hubo que cambiarlo por el cliente de Java 11 (`JdkClientHttpRequestFactory`), que no hace ese reintento.

7. Pruebas

44 pruebas en tres clases, con tres propósitos distintos.

Clase	N.º	Qué fija
`AuthenticationServiceTest`	24	Las reglas: qué se cuenta y qué se calla, cuándo se frena una cuenta, qué pasa con una sesión vencida. No dependen de la ruta HTTP
`AuthenticationControllerTest`	16	La traducción a HTTP: códigos de estado, cuerpos de error, cabeceras. Un 401 donde debía ir un 429 no se ve probando solo el servicio
`AuthenticationHttpTest`	4	Las condiciones de producción, sin transacción. Es la que encontró los dos fallos de §6

```
$ mvn test

Tests run: 24, Failures: 0, Errors: 0 — AuthenticationServiceTest
Tests run: 16, Failures: 0, Errors: 0 — AuthenticationControllerTest
Tests run:  4, Failures: 0, Errors: 0 — AuthenticationHttpTest

Tests run: 44, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

7.1 Las pruebas que valen por lo que impiden

- **«Da el mismo error si el correo no existe que si la contraseña no coincide».** Compara los dos mensajes. Si alguien «mejora la experiencia» separándolos, esta prueba se pone roja.
- **«Un correo repetido responde exactamente lo mismo que uno nuevo».** Compara los cuerpos byte por byte, no solo el código de estado.
- **«Con la cuenta frenada y la contraseña equivocada, no dice que está frenada».** Es el punto fino de §5.5, y es fácil de romper reordenando dos líneas.

- **«La respuesta no lleva el resumen de la contraseña».** Busca `$2a$` en el cuerpo completo, no solo en el campo esperado: así también detecta que aparezca donde nadie lo buscaba.
- **«Pasados los quince minutos, la cuenta vuelve a dejar entrar».** El tiempo entra como parámetro y no se lee del reloj dentro del servicio, así que la prueba tarda milisegundos en lugar de un cuarto de hora.

8. Verificación por HTTP real

```
$ curl -X POST localhost:8082/api/auth/register -H 'Content-Type: application/json' \
       -d '{"email":"caficultor@finca.co","password":"pergamino2026","fullName":"Luis Cardo-
na"}'
202  {"message":"Si el correo no estaba registrado, tu cuenta quedó creada. ..."}

$ curl -X POST localhost:8082/api/auth/login ...
200  {"message":"Autenticación satisfactoria.",
      "token":"QnqQHzHkblc1hdCmxVZ87FLbd-MQh_CZehXmmVgDKNU",
      "expiresAt":"2026-11-02T11:17:46Z",
      "account":{"id":2,"email":"caficultor@finca.co","role":"BUYER", ...}}

$ curl localhost:8082/api/auth/me -H "Authorization: Bearer $TOKEN"
200  {"id":2,"email":"caficultor@finca.co","fullName":"Luis Cardona","role":"BUYER", ...}

$ for i in 1 2 3 4 5; do curl -X POST .../login -d '{...,"password":"equivocada-123"}'; done
401  401  401  401  401

$ curl -X POST .../login -d '{...,"password":"pergamino2026"}'      # la BUENA
429  Retry-After: 899
     {"error":"account_locked","message":"Demasiados intentos fallidos.
      Vuelve a intentarlo en 15 minutos."}

$ curl -X POST .../login -d '{...,"password":"equivocada-123"}'     # la mala, ya frenada
401  {"error":"authentication_failed","message":"Correo o contraseña incorrectos."}
```

Y lo que quedó escrito en la base:

```
monkgrain_java=# SELECT email, left(password_hash,7), role, failed_attempts,
                        locked_until IS NOT NULL AS frenada
                 FROM auth_accounts;

           email            |  left   | role  | failed_attempts | frenada
----------------------------+---------+-------+-----------------+---------
 caficultor@finca.co        | $2a$10$ | BUYER |               5 | t
 ana.restrepo@finca.co      | $2a$10$ | BUYER |               0 | f
```

Dos cosas se comprueban de un vistazo: la contraseña no está —solo su resumen, con el algoritmo y el factor visibles— y el contador de intentos sí quedó guardado, que es lo que el defecto de §6.2 impedía.

9. Estándares aplicados

Estándar (AA1-EV02)	Cómo se cumple
Paquete `co.monkgrain.*`	`co.monkgrain.authentication`
Clases en `PascalCase`, métodos en `camelCase`	`AuthenticationService.requireAccount`
Constantes en `SCREAMING_SNAKE_CASE`	`MAX_FAILED_ATTEMPTS`, `LOCK_DURATION`
Código en inglés, comentarios en español	`failedAttempts`, con su porqué en español encima
Separación por capas	`controller → service → repository → model`, en una sola dirección
Rutas de API en plural y minúsculas	`/api/auth/...`
El comentario explica el porqué	«BCrypt y no SHA-256: un resumen rápido es un defecto aquí, no una virtud»
Nada de credenciales en el código	Variables de entorno con valor de respaldo para desarrollo

10. Cómo ejecutarlo

```
# 1. la base, si no existe
docker compose exec -T postgres psql -U monkgrain -d postgres \
  -c "CREATE DATABASE monkgrain_java;"
docker compose exec -T postgres psql -U monkgrain -d monkgrain_java \
  < src/main/resources/esquema.sql

# 2. pruebas
export MVN="$HOME/Desktop/PROYECTO MONK GRAIN/entregas_GA7/_herramientas/apache-ma-
ven-3.9.16/bin/mvn"
$MVN test

# 3. arrancar
$MVN package
java -jar target/autenticacion-pergamino.jar     # puerto 8082
```

11. Conclusión

El servicio hace lo que pide la evidencia: recibe usuario y contraseña, y responde «Autenticación satisfactoria.» o un error de autenticación. Lo que no es obvio en el enunciado es que **el error correcto es el que no explica nada**: cualquier detalle que distinga un correo registrado de uno que no lo está convierte el inicio de sesión en una herramienta de reconocimiento.

Y la lección de método, que vale más que el módulo: cuarenta pruebas verdes convivían con dos defectos graves —un endpoint que devolvía 500 y un freno que no frenaba— porque todas corrían dentro de una transacción que no existe en producción. Los encontró `curl`. La prueba que los habría encontrado existe ahora, y es la que corre sin transacción.